

4.6 Angular Services and Dependency Injection

4.6.1 Creating and Injecting Services, Using HttpClient to Call REST APIs

Core idea: Angular services are reusable classes used to share data, business logic, and functionality. Dependency Injection (DI) is Angular's mechanism for supplying a component or service with the dependencies it needs.

4.6 Angular Services

An Angular service is a class that contains reusable logic or data which can be shared by multiple parts of an application. Services are commonly used for data access, business logic, shared state, and communication with backend APIs.

Why use services?

- Code organization - keep business logic and data access outside components.
- Reuse - one service can be used by multiple components or other services.
- Maintainability - changes to shared logic can be made in one place.
- Dependency Injection - Angular can create and supply service instances where they are needed.
- Testing - isolated services are easier to test than logic mixed into a component.

Typical service structure

Basic service

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class MyService {

  getData() {
    return [];
  }
}
```

The `@Injectable` decorator marks the class as available for Angular's dependency-injection system. Using `providedIn: 'root'` makes the service available application-wide and is the common default for reusable services.

Creating a service with Angular CLI

Angular CLI

```
ng generate service my-service
# short form
ng g s my-service
```

The CLI creates a service file and its accompanying test file. The generated service can then be expanded with application-specific methods and state.

4.6.1 Dependency Injection

Dependency Injection is a design pattern in which an object receives the objects it depends on instead of creating those objects itself. Angular manages this process through its injector system.

Key concepts

Concept	Meaning
Dependency	An object a class needs to perform its work, such as a service or HttpClient.
Provider	Configuration that tells Angular how to create or supply a dependency.
Injector	Angular's mechanism that resolves providers and supplies dependencies.
Injection	The act of supplying a dependency to a class.
@Injectable	Decorator used to make a class suitable for Angular's DI system.
providedIn: 'root'	Registers the service with the root injector for application-wide use.

Why DI is useful

- Reduces direct object creation inside components.
- Encourages separation of responsibilities.
- Makes shared services available to many consumers.
- Supports testability because dependencies can be replaced or configured differently.

Providers and scopes

A dependency can be provided at different levels. A root provider is application-wide, while a provider configured on a component creates a service instance in that component's injection scope and its descendants.

Root provider

```
@Injectable({
  providedIn: 'root'
})
export class UserService {
  // one root-scoped service instance
}
```

Component-level provider

```
@Component({
  selector: 'app-example',
  providers: [UserService],
  template: `...`
})
export class ExampleComponent {
  // UserService is provided in this component scope
}
```

Injecting a Service into a Component

After a service is provided, a component can request it as a dependency. The course material demonstrates constructor injection; modern Angular also supports the `inject()` function.

Constructor-based injection

```
import { Component } from '@angular/core';
import { MyService } from './my-service';

@Component({
  selector: 'app-root',
  template: `
    <button (click)="addItem()">Add Item</button>
    <ul>
      <li *ngFor="let item of items">
        {{ item.value }}
      </li>
    </ul>
  `
})
export class AppComponent {
  items: any[] = [];

  constructor(private myService: MyService) {
    this.items = this.myService.getData();
  }

  addItem() {
```

```

this.myService.addData({
  id: Date.now(),
  value: 'New Item'
});

this.items = this.myService.getData();
}
}

```

inject() style

```

import { Component, inject } from '@angular/core';
import { MyService } from './my-service';

@Component({
  selector: 'app-root',
  template: `...`
})
export class AppComponent {
  private myService = inject(MyService);

  loadData() {
    return this.myService.getData();
  }
}

```

Remember: The course notes show constructor injection. The official Angular documentation also demonstrates the inject() function for current Angular applications.

Service Example: Shared Data

A service can keep data and provide methods such as getData() and addData(). The component uses the service rather than storing the shared data logic inside the component.

my-service.ts

```

import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class MyService {
  private data: any[] = [];

  getData() {
    return this.data;
  }
}

```

```

}

addData(item: any) {
  this.data.push(item);
}
}

```

app.component.ts

```

export class AppComponent {
  items: any[] = [];

  constructor(private myService: MyService) {
    this.items = this.myService.getData();
  }

  addItem() {
    const newItem = {
      id: Date.now(),
      value: 'New Item'
    };

    this.myService.addData(newItem);
    this.items = this.myService.getData();
  }
}

```

Flow

1. Angular creates or obtains the service instance from its injector.
2. The component receives the service as a dependency.
3. The component calls service methods such as `getData()` or `addData()`.
4. The service performs the shared logic and returns or updates data.
5. The component displays the returned data in its template.

4.6.2 HttpClient and REST APIs

HttpClient is Angular's service for communicating with HTTP backends. It supports common HTTP methods such as GET, POST, PUT, PATCH, and DELETE. HttpClient methods return RxJS Observables; the request is sent when the Observable is subscribed.

REST API basics

HTTP method	Typical purpose	Example
GET	Read data.	GET /api/products
POST	Create a new resource.	POST /api/products

PUT	Replace/update a resource.	PUT /api/products/10
PATCH	Partially update a resource.	PATCH /api/products/10
DELETE	Remove a resource.	DELETE /api/products/10

Setting up HttpClient

Current Angular applications commonly configure HttpClient with provideHttpClient() in application configuration.

Current Angular setup

```
import { ApplicationConfig } from '@angular/core';
import { provideHttpClient } from '@angular/common/http';

export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient()
  ]
};
```

Note: Older NgModule-based course examples may import HttpClientModule. In current Angular, provideHttpClient() is the standard application configuration approach.

Creating a Data Service with HttpClient

data.service.ts

```
import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

@Injectable({
  providedIn: 'root'
})
export class DataService {
  private apiUrl = 'https://api.example.com/posts';

  private http = inject(HttpClient);

  getPosts(): Observable<any[]> {
    return this.http.get<any[]>(this.apiUrl);
  }

  createPost(post: any): Observable<any> {
    return this.http.post<any>(this.apiUrl, post);
  }
}
```

```

}

updatePost(id: number, post: any): Observable<any> {
  return this.http.put<any>(`${this.apiUrl}/${id}`, post);
}

deletePost(id: number): Observable<any> {
  return this.http.delete<any>(`${this.apiUrl}/${id}`);
}
}

```

What the service methods do

- `getPosts()` sends a GET request and returns the server response as an Observable.
- `createPost(post)` sends a POST request with data in the request body.
- `updatePost(id, post)` sends a PUT request for a particular resource.
- `deletePost(id)` sends a DELETE request for a particular resource.

Important: The URL `https://api.example.com/posts` is a placeholder endpoint for learning. Replace it with the real URL of your backend REST API.

Using the Data Service in a Component

app.component.ts

```

import { Component, OnInit, inject } from '@angular/core';
import { DataService } from './services/data.service';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html'
})
export class AppComponent implements OnInit {
  posts: any[] = [];

  private dataService = inject(DataService);

  ngOnInit(): void {
    this.getPosts();
  }

  getPosts(): void {
    this.dataService.getPosts().subscribe({
      next: (data) => {
        this.posts = data;
      },
      error: (err) => {

```

```

        console.error('Error fetching posts:', err);
    }
});
}

addPost(): void {
    const newPost = {
        title: 'Angular',
        body: 'Learning HttpClient'
    };

    this.dataService.createPost(newPost).subscribe({
        next: (response) => {
            console.log('Post created:', response);
            this.getPosts();
        },
        error: (err) => {
            console.error('Error creating post:', err);
        }
    });
}
}

```

app.component.html

```

<div>
  <h2>Posts</h2>

  <ul>
    <li *ngFor="let post of posts">
      <strong>{{ post.title }}</strong>
      {{ post.body }}
    </li>
  </ul>

  <button (click)="addPost()">Add Post</button>
</div>

```

Request flow

6. The component calls a method of DataService.
7. DataService uses HttpClient to create the HTTP request.
8. The server processes the REST request.
9. HttpClient emits the response through the Observable.
10. The component's subscription handles the response and updates the view.

HTTP Request Options and Response Handling

HttpClient methods accept the request URL and an optional options object. Options can configure headers, query parameters, response handling, and other request behavior.

Headers and query parameters

```
import { HttpHeaders, HttpParams } from '@angular/common/http';

const headers = new HttpHeaders({
  'Content-Type': 'application/json'
});

const params = new HttpParams()
  .set('page', '1')
  .set('limit', '10');

this.http.get<any[]>('/api/posts', {
  headers,
  params
});
```

Basic error handling

Handling the Observable result

```
this.dataService.getPosts().subscribe({
  next: (data) => {
    this.posts = data;
  },
  error: (error) => {
    console.error('Request failed:', error);
  }
});
```

For larger applications, error handling can be centralized in an HTTP interceptor or a shared error-handling service. The basic subscribe({ next, error }) form is useful for understanding the request-response flow.

Typed responses

Typed HttpClient response

```
interface Product {
  id: number;
  name: string;
  price: number;
}

getProducts(): Observable<Product[]> {
```

```
return this.http.get<Product[]>(this.apiUrl);
}
```

Remember: The generic type supplied to HttpClient describes the expected response shape to TypeScript; it does not validate the server response at runtime.

Course-aligned REST API Example

The course material demonstrates creating a DataService, injecting HttpClient into the service, defining getPosts() and createPost() methods, injecting DataService into a component, subscribing to the returned Observable, and displaying the returned posts in the template.

Course-style service

```
// data.service.ts
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

@Injectable({
  providedIn: 'root'
})
export class DataService {
  private apiUrl = 'https://jsonplaceholder.typicode.com/posts';

  constructor(private http: HttpClient) {}

  getPosts(): Observable<any[]> {
    return this.http.get<any[]>(this.apiUrl);
  }

  createPost(post: any): Observable<any> {
    return this.http.post<any>(this.apiUrl, post);
  }
}
```

Course-style component

```
// app.component.ts
import { Component, OnInit } from '@angular/core';
import { DataService } from './services/data.service';

export class AppComponent implements OnInit {
  posts: any[] = [];
}
```

```

constructor(private dataService: DataService) {}

ngOnInit(): void {
  this.fetchPosts();
}

fetchPosts(): void {
  this.dataService.getPosts().subscribe(
    data => {
      this.posts = data;
    },
    error => {
      console.error('Error fetching posts:', error);
    }
  );
}

addPost(): void {
  const newPost = {
    title: 'New Post',
    body: 'Example body'
  };

  this.dataService.createPost(newPost).subscribe(
    response => {
      console.log('Post created:', response);
      this.fetchPosts();
    },
    error => {
      console.error('Error creating post:', error);
    }
  );
}

```

Course-style template

```

<div>
  <h2>Posts</h2>

  <ul>
    <li *ngFor="let post of posts">
      <strong>{{ post.title }}</strong>
      {{ post.body }}
    </li>
  </ul>

  <button (click)="addPost()">Add Post</button>
</div>

```

Quick Revision

Topic	Remember
Service	Reusable class for shared data, business logic, or API access.
@Injectable	Marks a class for Angular dependency injection.
providedIn: 'root'	Makes a service available through the root injector.
DI	Angular supplies required dependencies instead of the class creating them itself.
Injector	Resolves providers and creates/supplies dependencies.
HttpClient	Angular service used to communicate with HTTP/REST APIs.
Observable	HttpClient request methods return Observables.
GET	Read data.
POST	Create/send data.
PUT	Update/replace a resource.
DELETE	Delete a resource.
subscribe()	Starts an Observable request and handles emitted results.

Key code patterns

Task	Pattern
Create service	ng generate service service-name
Service decorator	@Injectable({ providedIn: 'root' })
Inject service	private service = inject(MyService)
Constructor injection	constructor(private service: MyService) {}
HttpClient injection	private http = inject(HttpClient)
GET	this.http.get<T>(url)
POST	this.http.post<T>(url, body)
PUT	this.http.put<T>(url, body)
DELETE	this.http.delete<T>(url)
Handle response	.subscribe({ next: ..., error: ... })

Service + DI + HttpClient flow

11. Create a reusable service.
12. Make it injectable and provide it.
13. Inject the service into the component or another service.
14. Inject HttpClient into the data service.
15. Write methods for GET, POST, PUT, PATCH, or DELETE.

16. Call the service method from the component.
17. Subscribe to the returned Observable and update the UI.

Exam memory: Keep API communication inside services when possible. Components should mainly coordinate the UI, while the service handles reusable data-access logic.